Product Requirements Document (PRD): Career-Ops

1. Product Overview

Career-Ops is an open-source, AI-powered job search automation pipeline designed to run entirely locally through any major AI coding CLI (Gemini CLI, Claude Code, OpenCode, etc.). Instead of treating the job search as a volume game, Career-Ops helps candidates systematically evaluate, tailor, and track high-quality applications. It flips the dynamic: companies use AI to filter candidates, and Career-Ops gives candidates AI to evaluate and choose companies.

1.1. Vision

To provide a private, locally-run command center that automates the tedious parts of the job search—resume tailoring, pipeline tracking, and offer evaluation—while maintaining high-quality, authentic applications.

2. Target Audience

- **Primary:** Software engineers, data scientists, product managers, and tech professionals.
- **Secondary:** Any professional who values tailored applications and wants to treat their job search as an organized, data-driven pipeline.

3. Core Features & Requirements

3.1. Onboarding & Configuration (Data Contract)

- **Requirement:** The system must strictly separate system files from user data.
- **Components:**
  - `cv.md` : Canonical resume source.
  - `config/profile.yml` : User details, target roles, salary expectations.
  - `modes/_profile.md` : User-specific archetypes and scoring weights.
  - `portals.yml` : Job board tracking configuration.
- **Flow:** If user data is missing on start, the AI must automatically enter an onboarding flow to generate these files.

3.2. Sourcing (Portal Scanning)

- **Requirement:** Ability to discover new job postings efficiently without excessive API costs.
- **Features:**
  - Zero-token scanning (`scan.mjs`) that directly queries ATS APIs (Greenhouse, Lever, Ashby).
  - Configurable target companies and role filters via `portals.yml` .
  - Deduping mechanism to track previously seen jobs (`scan-history.tsv`).
  - Output new jobs to a processing inbox (`data/pipeline.md`).

3.3. Job Evaluation Engine

- **Requirement:** Deep analysis of job descriptions to determine candidate fit.
- **Features:**
  - Extract JD using Playwright/WebFetch (verifying liveness).
  - **Six-Block Evaluation (A-F):**
    - **A:** Role Summary.

        - **B:** CV Match & Gap Mitigation.
        - **C:** Level Strategy.
        - **D:** Compensation Research.
        - **E:** CV Personalization Plan.
        - **F:** Interview Prep (STAR stories).
    - Generate a weighted 1-5 score across 10 dimensions.
    - **Rule:** If score < 4.0, explicitly advise the user against applying to save time.

3.4. Artifact Generation

- **Requirement:** Automatically generate customized application materials.
- **Features:**
    - Produce an ATS-optimized CV in PDF format (`generate-pdf.mjs` via Playwright converting HTML, or `generate-latex.mjs` for LaTeX/Overleaf).
    - Output comprehensive markdown reports (`reports/{id}-{slug}-{date}.md`).

3.5. Tracking & Orchestration

- **Requirement:** Maintain a single source of truth for the application pipeline.
- **Features:**
    - **Canonical Tracker:** `data/applications.md` tracks ID, date, company, role, score, status, and links to reports.
    - **Dashboard TUI:** A Go-based terminal UI (`dashboard/main.go`) to filter, sort, and view applications.
    - **Batch Processing:** Ability to run multiple evaluations concurrently using headless AI workers (`batch/batch-runner.sh`), tracked via state files and merged via `merge-tracker.mjs` .

3.6. Localization (Language Modes)

- **Requirement:** Support localized job markets.
- **Features:** Native language evaluation modes for English (default), German (DACH), French, Japanese, and Turkish, parsing local labor terminology natively.

3.7. Continuous Learning & Interview Prep

- **Requirement:** The system must improve as the user interviews.
- **Features:**
    - `interview-prep/story-bank.md` to accumulate behavioral stories.
    - Follow-up cadence calculation (`followup-cadence.mjs`) to prompt users when to email recruiters.

4. Architecture & Technical Stack

- **Interface:** Any AI Coding CLI supporting standard agent skills.
- **Scripts:** Node.js (`.mjs`) for validation, merging, PDF generation, and scanning.
- **Web Extraction:** Playwright (preferred for dynamic sites/liveness checks) and basic WebFetch.
- **Dashboard:** Go (Terminal UI).
- **Data Storage:** Markdown, YAML, and TSV (local files, no database required).
- **CI/CD:** GitHub Actions for testing (`test-all.mjs`), versioning, and branch protection.

5. Non-Functional Requirements

- **Privacy:** 100% local. User data (`data/*` , `cv.md` , `config/profile.yml`) is gitignored.
- **Extensibility:** Users must be able to modify templates, scoring weights, and agent instructions without breaking system updates.
- **Ethical Constraints:** The AI must *never* auto-submit applications. It drafts and prepares; the human always clicks apply. Focus is strictly on quality over spam.

6. Future Considerations

- Expand ATS integrations for scanning (e.g., Workday).
- Enhance the Go Dashboard TUI with more analytics.
- Integration with email clients for automated follow-up drafting.